МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
ФАКУЛЬТЕТ ІНФОРМАТИКИ ТА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ
КАФЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

Лабораторна робота №2

з дисципліни «Знайомство із середовищем розробки та складання модульних проектів програм»

на тему ***«*Розробка графічного редактора об'єктів*»***

Виконала:
студентка групи ІМ-51
Господінова М.М.
Перевірив:
Рекечинський Дмитро Олександрович

Київ 2026

Лабораторна робота №2. Розробка графічного редактора об’єктів
Мета: отримати вміння та навички використовувати інкапсуляцію, абстракцію типів, успадкування та поліморфізм на основі класів, запрограмувавши простий графічний редактор в об’єктно-орієнтованому стилі.
Завдання:
1. Створити у середовищі розробки новий проект з ім’ям Lab2.
2. Скомпілювати проект і отримати виконуваний файл програми.
3. Перевірити роботу програми. За потреби налагодити програму.
4. Проаналізувати та прокоментувати результати та вихідний текст програми.
5. Оформити звіт.
Варіант 6:
Кількість об’єктів - 106
Тип масиву - динамічний
“Гумовий” слід - суцільний синій
Введення прямокутника - за двома протилежними кутами
Прямокутник - чорний контур, жовте заповнення
Введення еліпса - від центра до кута охоплюючого прямокутника
Еліпс - чорний контур, сіре заповнення
Позначення типу фігури - у меню

Вихідний код
main.py

```
import tkinter as tk
from tkinter import messagebox

from shapes import Point, Line, Rectangle, Ellipse



class GraphicEditor:
   def __init__(self, root):
       self.root = root
       self.root.title("Lab2 — Graphic Object Editor")
       self.root.geometry("900x600")

       self.shapes = []
```

```
        self.max_shapes = 106

        self.current_shape = "Point"
        self.drawing_shape = self.current_shape

        self.start_x = 0
        self.start_y = 0

        self.preview_id = None

        self.create_menu()
        self.create_canvas()
        self.bind_mouse_events()

    def create_menu(self):
        menu_bar = tk.Menu(self.root)

        file_menu = tk.Menu(menu_bar, tearoff=0)
        file_menu.add_command(label="Clear",
command=self.clear_canvas)
        file_menu.add_separator()
        file_menu.add_command(label="Exit",
command=self.root.destroy)
        menu_bar.add_cascade(label="File", menu=file_menu)

        self.objects_menu = tk.Menu(menu_bar, tearoff=0)

        self.shape_types = ["Point", "Line", "Rectangle",
"Ellipse"]

        self.shape_var = tk.StringVar(value=self.current_shape)

        for shape_name in self.shape_types:
            self.objects_menu.add_radiobutton(
                label=shape_name,
                variable=self.shape_var,
```

```
                value=shape_name,
                command=self.select_shape,
            )

        menu_bar.add_cascade(label="Objects",
menu=self.objects_menu)

        help_menu = tk.Menu(menu_bar, tearoff=0)
        help_menu.add_command(label="About",
command=self.show_about)
        menu_bar.add_cascade(label="Help", menu=help_menu)

        self.root.config(menu=menu_bar)

    def create_canvas(self):
        self.canvas = tk.Canvas(self.root, bg="white",
cursor="cross")
        self.canvas.pack(fill=tk.BOTH, expand=True)

    def select_shape(self):
        self.current_shape = self.shape_var.get()

    def bind_mouse_events(self):
        self.canvas.bind("<Button-1>", self.on_mouse_down)
        self.canvas.bind("<B1-Motion>", self.on_mouse_move)
        self.canvas.bind("<ButtonRelease-1>", self.on_mouse_up)

    def on_mouse_down(self, event):
        self.start_x = event.x
        self.start_y = event.y
        self.drawing_shape = self.current_shape

        if self.drawing_shape == "Point":
            if len(self.shapes) >= self.max_shapes:
                messagebox.showwarning(
                    "Warning",
```

```
                    f"The limit of {self.max_shapes} objects has
been reached.",
                )
                return

            shape = Point(event.x, event.y)
            self.shapes.append(shape)
            shape.draw(self.canvas)

    def on_mouse_move(self, event):
        if self.drawing_shape == "Point":
            return

        self.canvas.delete("preview")

        self.draw_preview(event.x, event.y)

    def on_mouse_up(self, event):
        self.canvas.delete("preview")

        if self.drawing_shape == "Point":
            return

        if len(self.shapes) >= self.max_shapes:
            messagebox.showwarning(
                "Warning", f"The limit of {self.max_shapes} objects
has been reached."
            )
            return

        x1 = self.start_x
        y1 = self.start_y
        x2 = event.x
        y2 = event.y

        if self.drawing_shape == "Line":
```

```
            shape = Line(x1, y1, x2, y2)

        elif self.drawing_shape == "Rectangle":
            shape = Rectangle(x1, y1, x2, y2)

        elif self.drawing_shape == "Ellipse":
            shape = Ellipse(x1, y1, x2, y2)

        else:
            return

        self.shapes.append(shape)
        shape.draw(self.canvas)

    def draw_preview(self, x, y):
        x1 = self.start_x
        y1 = self.start_y

        if self.drawing_shape == "Line":
            self.canvas.create_line(x1, y1, x, y, fill="blue",
width=2, tags="preview")

        elif self.drawing_shape == "Rectangle":
            self.canvas.create_rectangle(
                x1, y1, x, y, outline="blue", fill="", width=2,
tags="preview"
            )

        elif self.drawing_shape == "Ellipse":
            left = min(x1, 2 * x1 - x)
            top = min(y1, 2 * y1 - y)
            right = max(x1, 2 * x1 - x)
            bottom = max(y1, 2 * y1 - y)

            self.canvas.create_oval(
                left,
```

```
                top,
                right,
                bottom,
                outline="blue",
                fill="",
                width=2,
                tags="preview",
            )

    def clear_canvas(self):
        self.shapes.clear()
        self.canvas.delete("all")
        self.preview_id = None

    def show_about(self):
        messagebox.showinfo(
            "About",
            "Lab2 — Graphic Object Editor.\n"
            "Developed using Python and Tkinter.",
        )


if __name__ == "__main__":
    root = tk.Tk()
    app = GraphicEditor(root)
    root.mainloop()
```

shapes.py

```
from abc import ABC, abstractmethod


class Shape(ABC):
    @abstractmethod
    def draw(self, canvas):
        pass
```

```
class Point(Shape):
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def draw(self, canvas):
        radius = 3
        canvas.create_oval(
            self.x - radius,
            self.y - radius,
            self.x + radius,
            self.y + radius,
            fill="black",
            outline="black",
        )


class Line(Shape):
    def __init__(self, x1, y1, x2, y2):
        self.x1 = x1
        self.y1 = y1
        self.x2 = x2
        self.y2 = y2

    def draw(self, canvas):
        canvas.create_line(self.x1, self.y1, self.x2, self.y2,
fill="black", width=2)


class Rectangle(Shape):
    def __init__(self, x1, y1, x2, y2):
        self.x1 = x1
        self.y1 = y1
        self.x2 = x2
        self.y2 = y2
```

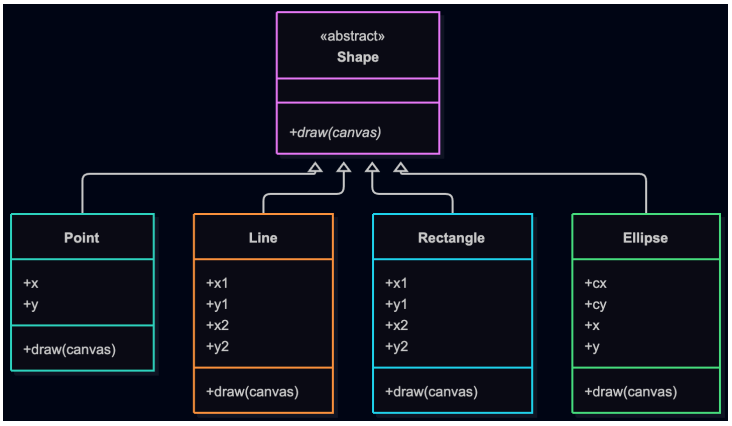
```
    def draw(self, canvas):
        canvas.create_rectangle(
            self.x1, self.y1, self.x2, self.y2, outline="black",
fill="yellow", width=2
        )


class Ellipse(Shape):
    def __init__(self, cx, cy, x, y):
        self.cx = cx
        self.cy = cy
        self.x = x
        self.y = y

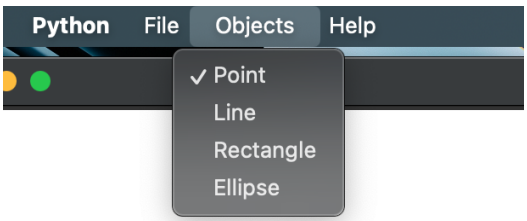
    def draw(self, canvas):
        x1 = min(self.cx, 2 * self.cx - self.x)
        y1 = min(self.cy, 2 * self.cy - self.y)
        x2 = max(self.cx, 2 * self.cx - self.x)
        y2 = max(self.cy, 2 * self.cy - self.y)

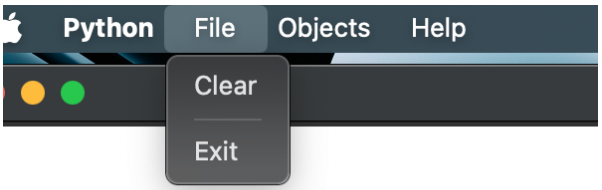
        canvas.create_oval(x1, y1, x2, y2, outline="black",
fill="gray", width=2)
```

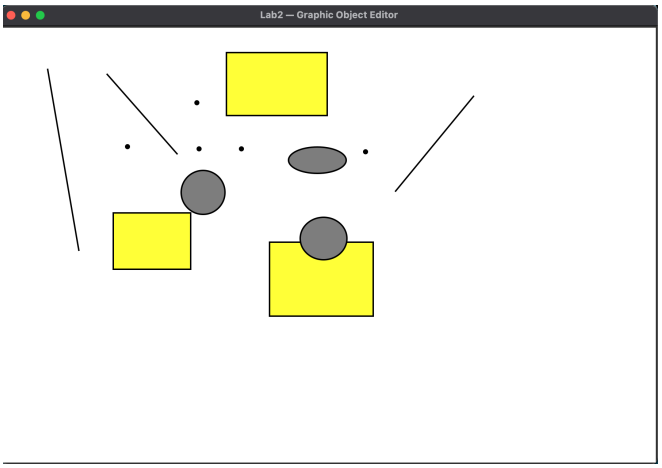
Схема успадкування класів:



Тестування (скріншоти)









Висновок:

У ході виконання лабораторної роботи № 2 було розроблено графічний редактор геометричних об’єктів із використанням мови програмування Python та бібліотеки Tkinter.

У програмі реалізовано абстрактний клас Shape та класи Point, Line, Rectangle і Ellipse, які успадковують його та реалізують власний метод draw(). Це дало змогу застосувати принципи абстракції, успадкування та поліморфізму.

Було реалізовано графічний інтерфейс, меню вибору типу об’єкта, обробку подій миші, попередній перегляд фігур під час малювання та очищення полотна.

У результаті роботи отримано практичні навички об’єктно-орієнтованого програмування та створення графічних застосунків.